
Abbreviations]VS, virtual screening

Application Notes

APPLICATION NOTES

Ringtail: an upgraded tool for managing molecular docking and virtual screening data at scale

May-Linn Paulsen ^{1,*}

¹Center for Computational Structural Biology, The Scripps Research Institute, 10550 North Torrey Pines Road, La Jolla 92037, CA, USA

*Corresponding author. mpaulsen@scripps.edu

Abstract

Virtual screening campaigns now routinely dock libraries of millions of compounds, generating pose and interaction data at a scale where downstream analysis becomes the bottleneck. Ringtail [Hansel-Harris et al., 2023] addresses this by organizing molecular docking results into a compact, queryable database and providing fast, flexible filtering, clustering, and export through both a command-line interface and a Python API. We present Ringtail 3, a major upgrade on two fronts. First, a new DuckDB storage backend, together with a re-engineered parsing and storage pipeline, filters large databases substantially faster than the original SQLite backend while producing smaller files. Second, Ringtail now reads output from across the AutoDock suite—AutoDock-GPU, AutoDock-Vina, and the new high-throughput AutoDock-6 (AD6) engine [?]¹—and computes receptor interactions internally. Ringtail 3 is freely available at <https://github.com/forlilab/Ringtail> under the [LICENSE] license and is tested on Linux, macOS, and Windows.

Key words: virtual screening, molecular docking, AutoDock, AutoDock-6, DuckDB

Key words: [

Introduction

Virtual screening (VS) is a cornerstone of computational drug discovery, and campaigns now routinely evaluate libraries of millions of molecules. A single campaign can produce tens of millions of docked poses together with their predicted receptor interactions, and identifying promising candidates within this volume—filtering by docking score, ligand efficiency, specific receptor interactions, substructure, or three-dimensional placement, then clustering and exporting the survivors—has become a major bottleneck. Ringtail was introduced to address exactly this: it deposits docking results into a unified, queryable database and exposes an extensive set of predefined filters, clustering, and export options through both a user-friendly command-line tool and a Python API [Hansel-Harris et al., 2023].

Here we present Ringtail 3, which substantially extends both the performance and the scope of the tool. Filtering at scale is

accelerated by a new DuckDB storage backend and a re-engineered parsing, storage, and bookmarking pipeline; input support now spans the full AutoDock suite, including the new AutoDock-NG engine; and additional export options make it easier to share curated results with collaborators.

Features

Ringtail 3 retains the full filtering, clustering, and export functionality of the original release while substantially improving performance and broadening the range of supported docking engines. The principal new capabilities are described below.

Faster analysis at scale

Ringtail 3 introduces DuckDB as a storage backend alongside the original SQLite. DuckDB is an embedded, serverless analytical

database with columnar storage optimized for the read-heavy aggregation queries that filtering a docking database entails. It is selected at database creation with `--storage_type duckdb` and auto-detected thereafter, requiring no change to the user-facing API, and write times remain comparable to SQLite. Interaction-based filtering, however, is markedly faster and the resulting databases are smaller: for a screen of roughly two million ligands, a combined docking-score-and-interaction filter completes in about 2.6s with DuckDB versus about 12s with SQLite, in a 9 GB versus 14 GB database. These gains are compounded by a re-engineered storage layer: docking results are now buffered and committed in large batches rather than individually, and filtered result sets are stored in dedicated `Filters` and `Filtered_poses` tables rather than as SQL views, which markedly speeds up repeated and progressive filtering.

Broader docking-engine support

Ringtail 3 reads results from across the AutoDock suite: Docking Log Files (DLGs) from AutoDock-GPU, PDBQT output from AutoDock-Vina, and SDF output from the new high-throughput AutoDock-6 (AD6) engine [?]. All engines populate the same unified schema, so users select their source with `--docking_mode` and otherwise interact with the database identically. Receptor–ligand interactions are now computed within Ringtail directly from the stored receptor, and can be (re-)calculated after the fact—useful when interactions were not stored at database creation, or when a different interaction-distance cutoff is desired.

Interactive graphical interface

Ringtail 3 is accompanied by an interactive graphical interface for exploring filtered results: a scatter plot of docking score versus ligand efficiency with click-to-view three-dimensional poses, and a status-triage workflow for tagging compounds as accepted, maybe, or rejected, persisted directly in the database. It replaces the previous static PyMOL/matplotlib plotting utilities.

Additional analysis and workflow enhancements

Further additions include `merge_databases` for combining databases from screening campaigns run in parallel; clustering by Morgan or interaction fingerprint over any saved selection; new molecular-weight filters; and several CSV export modes that let users write exactly the columns they need without writing SQL.

Implementation

Ringtail is implemented in Python (≥ 3.9) and distributed through PyPI and conda-forge. The database schema is defined in a single declarative module that emits the appropriate SQL for either backend, and result-file parsing is parallelized across available CPU cores. The same functionality is exposed through a command-line interface (`rt_process_vs` and companion tools) and a documented Python API, so Ringtail can be used interactively or embedded in automated screening pipelines. Core dependencies include RDKit, NumPy, SciPy, pandas, and Meeko.

Availability and Implementation

Ringtail is freely available at <https://github.com/forlilab/Ringtail> under the [LICENSE] license. Requires Python ≥ 3.9 and is tested on Linux, macOS, and Windows. Installation via conda-forge (`conda install -c conda-forge ringtail`) resolves all dependencies and is recommended; a PyPI package is also available for users who prefer to manage dependencies themselves.

Acknowledgements

...

Funding

...

References

Althea T. Hansel-Harris, Diogo Santos-Martins, Niccolò Bruciaferri, Andreas F. Tillack, Matthew Holcomb, and Stefano Forli. Ringtail: A Python tool for efficient management and storage of virtual screening results. *Journal of Chemical Information and Modeling*, 63(7):1858–1864, 2023. doi: 10.1021/acs.jcim.3c00166.